This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/TQE.2020.doi

Application of Quantum Recurrent Neural Network in Low Resource Language Text Classification

WENBIN YU[1,2,4](Member,IEEE), LEI YIN[1], CHENGJUN ZHANG[2,3,4](Member,IEEE), YADANG CHEN[3](Member,IEEE), ALEX X. LIU[5](Fellow,IEEE)

[1]School of Software, Nanjing University of Information Science and Technology, Nanjing 210044,China; ywb@nuist.edu.cn(W.Y.)
[2]Nanjing University of Information Science & Technology, Wuxi Institute of Technology, Jiangsu Wuxi 214000, China
[3]School of Computer Science, Nanjing University of Information Science and Technology, Nanjing 210044, China; adamchen@nuist.edu.cn(Y.C.)
[4]Jiangsu Collaborative Innovation Center of Atmospheric Environment and Equipment Technology (CICAEET), Nanjing University of Information Science and Technology, Nanjing 210044, China
[5]Shandong Provincial Key Laboratory of Computer Networks, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan 250014, China; alexliu360@gmail.com

Corresponding author:zhangcj5@gmail.com (C.Z.).

This research was funded by the Natural Science Foundation of China, grant numbers 62071240; the Natural Science Foundation of Jiangsu Province, grant number BK20231142; Innovation Program for Quantum Science and Technology, grant number 2021ZD0302900.

ABSTRACT

Text sentiment analysis is an important task in natural language processing and has always been a hot research topic. However, in low-resource regions such as South Asia, where languages like Bengali are widely used, the research interest is relatively low compared to high-resource regions due to limited computational resources, flexible word order, and high inflectional nature of the language. With the development of quantum technology, quantum machine learning models leverage the superposition property of qubits to enhance model expressiveness and achieve faster computation compared to classical systems. To promote the development of quantum machine learning in low-resource language domains, we propose a quantum-classical hybrid architecture. This architecture utilizes a pre-trained multilingual BERT model to obtain vector representations of words and combines the proposed Batched Upload Quantum Recurrent Neural Network (BUQRNN) and Parameter Non-shared Batched Upload Quantum Recurrent Neural Network (PN-BUQRNN) as feature extraction models for sentiment analysis in Bengali. Our numerical results demonstrate that the proposed BUQRNN structure achieves a maximum accuracy improvement of 0.993% in Bengali text classification tasks while reducing average model complexity by 12%. The PN-BUQRNN structure surpasses the BUQRNN structure once again and outperforms classical architectures in certain tasks.

INDEX TERMS Natural language processing,Quantum machine learning,Quantum recurrent neural network.

I. INTRODUCTION

As one of the classical subfields of machine learning [1]–[3], natural language processing (NLP) [4], [5] has been a hot research topic in recent years. Text sentiment analysis (SA) [6], as a subtask of NLP, aims to classify text into positive and negative sentiment categories by detecting the polarity of the text. SA has been applied in various domains, including lexicon-based SA [7], machine learning-based SA [8], and deep learning-based SA [9]. Remarkable results have been achieved in SA for high-resource languages such as English and Chinese [10], [11]. However, due to the complexity of language grammar, limited usage, and expensive computational resources, SA in low-resource languages has not been extensively explored. With the development of the internet, a large influx of textual comments has made SA in low-resource languages feasible. In general, effective SA tasks can be achieved by combining good word embedding models with efficient feature extraction models. In the case of studying word embeddings for Bengali texts, a significant challenge lies in capturing the rich expressions of sentiment present in the Bengali language. Due to the complexity of grammar rules, the extraction of features for sentiment clas-

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903

sification becomes intricate as sentiment information may be expressed differently in sentences. The emergence of pre-trained language models [12], [13] has improved feature extraction in Bengali sentiment classification tasks, as they can effectively capture sentiment information within Bengali sentences by learning rich language representations and contextual understanding.

Despite the positive role of pre-trained language models in Bengali sentiment classification tasks, traditional feature extraction models still face efficiency challenges. Quantum deep learning [14], [15] combines the concepts of quantum computing and deep learning, leveraging the parallelism advantage [16] of quantum computing to accelerate the training and inference processes of models. By utilizing quantum neural networks and quantum gate operations, quantum deep learning models can handle complex sentiment classification tasks more efficiently [17]. A quantum-classical hybrid recurrent neural network model (QRNN) based on the quantum variational circuit (VQC) core was proposed in literature [18]. Such networks have been successfully applied as feature extractors in text classification tasks for high-resource languages, demonstrating better performance compared to their classical counterparts. Considering the characteristics of low-resource languages, this sparks the idea of using quantum algorithms to improve low-resource text SA tasks. However, previous studies have shown that QRNNs may struggle to effectively capture semantic information and may even result in information loss when dealing with longer sequences. The current challenges can be summarized as follows:

- Due to the limitations of current Noisy Intermediate-Scale Quantum (NISQ) devices [19], it is necessary to match the dimensionality of the input sequence with the number of qubits. Previous QRNN models employed parameter-sharing linear layers to reduce the dimensionality of the input data, which may potentially result in the loss of semantic information to some extent.
- Previous QRNN models did not optimize the Quantum Variational Circuit (VQC) specifically but instead utilized parameter-sharing linear layers for optimization across all VQCs.

In response to the first situation mentioned above, we designed and utilized a Batched Uploading Quantum Neural Network (BUQNN), which is essentially a structure that incorporates VQC circuits. The BUQNN divides the input feature sequence into batches and loads them into the circuit to obtain the complete semantic information. We refer to the QRNN model that utilizes this BUQNN as BUQRNN. By adopting this approach, we can alleviate the semantic information loss caused by previous methods with a S number of qubits.

Regarding the second situation mentioned, in reference [20], a non-parameter-sharing linear layer was applied after the VQC to enhance the expressiveness of the circuit. We followed this idea and made improvements by using non-parameter-sharing linear layers both before and after each VQC. This allows for independent optimization of each VQC, which is advantageous for the model.

Our contributions are as follows:

- We proposed a Batched Uploading Quantum Recurrent Neural Network (BUQRNN) specifically designed for sequential data. This method requires only a S number of qubits and mitigates the loss of semantic information caused by previous approaches.
- We introduced a Parameter Non-sharing BUQRNN (PN-BUQRNN) that employs independent linear layers for each VQC, enabling independent optimization of each VQC in the model structure.
- For the first time, we applied quantum algorithms to address sentiment classification tasks in low-resource languages such as Bengali, which holds significant importance for advancing the development of quantum in low-resource languages.

The rest of the paper is organized as follows. Section 2 focuses on the text classification process and discusses word embedding techniques and the QRNN network for low-resource language text classification. Section 3 presents the specific implementation approach to address the aforementioned issues. Section 4 describes the numerical simulation results, and Section 5 concludes the paper, providing insights into future directions and prospects.

II. RELATED WORK

Reference [21] employed a combination of Multilingual BERT embeddings and RNN for text classification tasks in Bengali. In this section, we will introduce an improved quantum-classical hybrid model based on its architecture. We will discuss various methods for text classification in the low-resource language domain and provide an overview of Bengali-specific word embedding techniques and the QRNN model.

A. WORD EMBEDDING

In the domain of high-resource languages, there are generally two methods for generating word vectors: contextual word embedding techniques and non-contextual word embedding techniques. BERT as a context-based pre-trained language model, can also be employed for word embedding. In the reference [22], BERT is utilized to obtain word vectors for English text, followed by feature extraction using the quantum TCN. The results demonstrate the high performance of BERT while also indicating the feasibility of related quantum models in text classification tasks. Similar to the high-resource language domain, low-resource languages can be categorized into two main types in terms of word embedding methods. The first type is non-contextual word embedding methods. Reference [23] used the word2vec model to generate Bengali word vectors. However, word2vec does not handle subword information, such as affixes, and does not consider spelling characteristics of words. In contrast to Word2Vec, GloVe [24]

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903

considers both local context and global statistical information to generate word vectors. FastText [25], an open-source word embedding and text classification library developed by Facebook AI Research, differs from Word2Vec and GloVe in that it considers subword information within words. In contrast to non-contextual word embeddings, the second type, contextual word embeddings, take into account the context of words in specific sentences or texts, enabling better capture of word semantics. Multilingual BERT (MBERT) [26], as a multilingual version of the BERT model, has been pretrained on Wikipedia texts from 104 languages. An important feature of MBERT is its bidirectional self-attention mechanism using the Transformer model, which allows for a better understanding of the complex semantics of words within their context. Experimental results [21] demonstrate that compared to traditional word embeddings such as FastText and GloVe, MBERT performs better in Bengali word embeddings. In addition to employing classical methods for text word embedding as mentioned above, the reference [27] proposes a novel technique for text word embedding using a quantum language model. The results indicate that word vectors mapped by the quantum language model can achieve performance comparable to their classical counterparts in downstream tasks. In this paper, we will choose MBERT as the word embedding model for Bengali text. The sentence vector x can be represented as $\mathbf{x} = (x_1, x_2, x_3, \ldots, x_t)$ after tokenization, where x_t represents the t-th word in the sentence. After passing through the MBERT model, the corresponding BERT vector representation can be obtained as follows:

$$\mathbf{MBERT}(x_1, x_2, x_3, \ldots, x_t) = (e_1, e_2, e_3, \ldots, e_t) \quad (1)$$

Here, each e_t represents the MBERT vector representation of the sentence with index j at time t.

B. QRNN

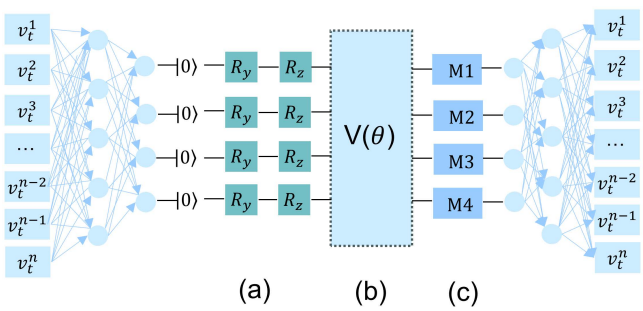


FIGURE 1. The VQC structure used in QLSTM consists of (a) an encoding layer, (b) a variational layer, and (c) a measurement layer. The encoding layer is composed of a series of rotation gates used for encoding.

QLSTM [28] is a variation of QRNN that incorporates a variational quantum circuit (VQC) [29] to construct quantum-gated units. The VQC adjusts the initial states of qubits through a series of rotation gates, performs quantum entanglement operations using a series of CNOT gates [30], and finally collapses them into classical states through measurement operations. The parameters of the rotation gates need to be updated through gradient descent. Fig.1 illustrates the structure of the variational quantum circuit included in the QLSTM network. In this case, we assume the circuit is simulated with 4 qubits. The input word vector sequence $\vec{v_t} = (v_t^1, v_t^2, ..., v_t^n)$, where $\vec{v_t}$ is composed of the current MBERT output e_t and the previous hidden layer feature h_{t-1}. Initially, a linear layer is used to match the number of qubits, and the encoding layer utilizes angle encoding [31] to embed the input sequence into the circuit. The variational layer optimizes the rotation angles of the qubits on the Bloch sphere [32] using a series of rotation gates with updatable parameters. The measurement part employs Pauli-Z gates to measure the states of the qubits. Measuring in the Pauli-Z basis $\sigma_z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ means projecting the state onto one of the eigenstates of the Pauli-Z matrix, namely $|0\rangle$ or $|1\rangle$. The measured classical bits are then expanded to match the size of the hidden layer through another linear layer. However, when $I > n$ (where n is the number of qubits and I is the length of the feature sequence), the current approach typically reduces the input dimensionality to match the number of qubits through a linear layer, which may result in the loss of semantic information to some extent. We draw inspiration from a quantum algorithm for image classification on the MNIST handwritten dataset [33], which differs from the previous data re-uploading approach [34]. The former method slices the image horizontally and uploads the value of each pixel to the quantum circuit, while the latter repeatedly uploads the feature vector to the circuit. Due to the limitations of current NISQ devices, the latter approach requires significant computational resources for processing high-dimensional feature sequences, making it difficult to implement. The former approach, by batch uploading, reduces the dependence on the number of qubits and fully loads the feature sequence into the circuit. We aim to address the first issue mentioned in the introduction of traditional QRNN using this method.

In addition, the parameters of the linear embedding layer before each VQC and the linear expansion layer after each VQC in traditional QRNN are shared. Taking QLSTM as an example in reference [28], the model uses four VQCs to replace the classical network layers, and these four VQCs share the embedding layer and expansion layer. However, this shared linear layer cannot be effectively optimized for different VQCs. In reference [20], non-shared linear layers with separate parameters were added after each VQC in QLSTM to achieve independent optimization of the circuits. We were inspired by this and made improvements by incorporating non-shared parameter linear layers before and after each VQC, along with the aforementioned batch uploading approach, to achieve more comprehensive circuit optimization and further improve the performance of QRNN.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903

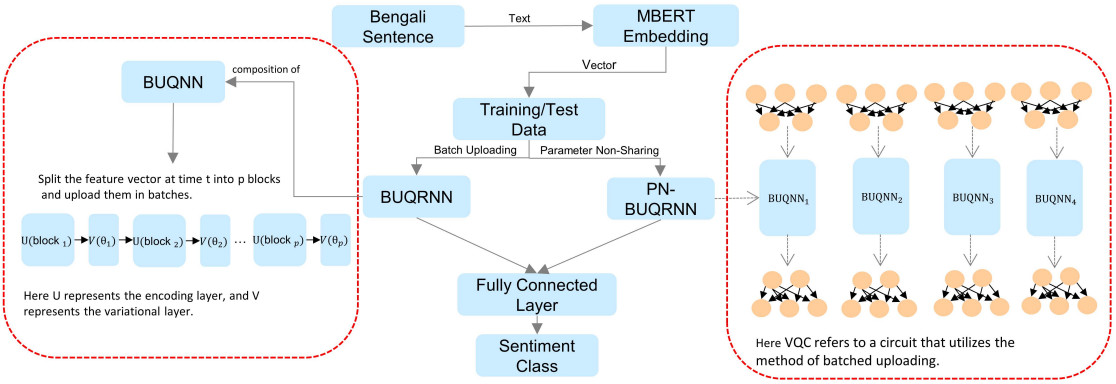


FIGURE 2. The framework includes two red dashed sections, showcasing the proposed structures designed to address the mentioned issues.

III. THE QRNN WE PROPOSE

Regarding the first issue mentioned in the introductory section, in sections A and B, we demonstrate the replacement of classical neural networks in classical RNN networks with a Batched Uploading Quantum Neural Network (BUQNN) and apply it to QRNN, proposing a BUQRNN based on a classical-quantum hybrid framework. Unlike traditional QRNN, which requires matching the number of qubits to the feature vectors, our approach provides universality and effectiveness for handling higher-dimensional feature vectors. BUQNN divides input features into batches according to a predefined number of qubits and passes them through a variational quantum circuit, forming an n-layer encoding-variational hybrid structure. This approach allows for processing sequence data without reducing the dimensionality of feature vectors and does not require a large number of qubits to handle the data. Addressing the second issue mentioned in the introductory section, section C provides a specific solution. The workflow for the sentiment classification task in Bengali language is shown in Fig. 2. The input text is transformed into word embeddings using MBERT, and then the word vectors are fed into the proposed BUQRNN or PN-BUQRNN for feature extraction. Finally, the extracted features are input into a fully connected layer for classification. We will now describe the implementation details of BUQRNN and PN-BUQRNN.

A. BATCH UPLOADING QUANTUM NEURAL NETWORK

In the QNN, the encoding gate, decoding gate, and variational gate are further divided into encoding layers, decoding layers, and variational layers. The selection of encoding gates is based on the chosen encoding method and the number of input features. The optimal choice of encoding method is crucial for successful learning of the QNN model. We implement the BUQNN using a multi-layer encoding-variational structure. The left portion of Fig.2 illustrates the BUQNN structure that we employ. We divide the features $\vec{v_t} = (v_t^1, v_t^2, ..., v_t^n)$ into batches based on the number of qubits, such that the number of batches is $n/N = p$. Here, N represents the number of qubits, and n represents the dimensionality of the input word vector features. This assumes a scenario where the division is evenly divisible. If it is not divisible, the number of batches p needs to be increased by 1, and the remaining space is padded with zero elements. After the division, $\vec{v_t} = (\vec{batch}_1, \vec{batch}_2, ..., \vec{batch}_p)$, where $\vec{batch}$ is a vector containing N features. In this demonstration, we use four qubits, and there is one variational layer between any two encoding layers. This restriction is just a design choice, and alternative design schemes can be chosen. Each feature batch undergoes angle encoding embedding circuit and then a variational layer. One feature batch is uploaded to the circuit at a time, and after loading p times, all feature vectors can be embedded in the circuit. It is important to note that the essence of BUQNN is a VQC using batch uploading, which differs from the structure of the VQC included in traditional QLSTM networks. It can directly embed feature vectors into the circuit without the need for additional linear layers. The BUQNN with linear layers will be mentioned in Section D of this chapter.

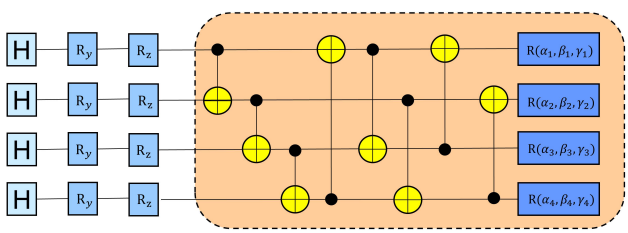


FIGURE 3. The encoding layer circuit and variational layer structure that we use.

Fig.3 illustrates the encoding layer circuit and variational layer structure that we utilize. In the simulation with n qubits, let's consider the example of a batch vector $\vec{batch}1 = (v_t^1, v_t^2, ..., v_t^N)$ where $1 \leq i \leq N$. For each v_t^i, we generate angles $\theta_{i,1} = arctan(v_t^i)$ and $\theta_{i,2} = arctan(v_t^{i^2})$, resulting in a total of 2i rotation angles. $\theta_{i,1}$ is applied using the $Ry(\theta_{i,1})$ gate for rotation around the y-axis, while $\theta_{i,2}$ is applied using the $Rz(\theta_{i,2})$ gate for rotation around the z-axis. The encoded data is in a quantum state and undergoes a series of unitary operations, including multiple CNOT gates and

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903

single-qubit rotation gates. $R(\theta, \alpha, \gamma)$ represents a general parameterized single-qubit rotation gate and can be expressed as:

$$\mathbf{R}(\theta_1, \theta_2, \theta_3) = \begin{pmatrix} e^{i\theta_2}\cos(\theta_1) & e^{i\theta_3}\sin(\theta_1) \\ -e^{i\theta_3}\sin(\theta_1) & e^{-i\theta_2}\cos(\theta_1) \end{pmatrix}$$

Assuming a quantum circuit consists of 4 qubits.The aforementioned process of quantum state changes can be summarized as follows:

- First, the initial state of the four qubits in the quantum circuit is denoted as $|\psi_0\rangle$, which is typically a pure state with all qubits in the ground state $|0\rangle$, i.e., $|\psi_0\rangle = |0000\rangle$.
- Next, $\vec{batch}_1$ is encoded into $|\psi_0\rangle$ to form a new quantum state $|\psi_1\rangle$. The encoding operation uses R_Y and R_Z gates, the rotation angles of which are determined by the elements in $\vec{batch}_1$. We denote this operation as $U(\vec{batch}_1)$, so the quantum state becomes $|\psi_1\rangle = U(\vec{batch}_1)|\psi_0\rangle$.
- In the third step, additional quantum operations, $V(\theta_1)$, are applied to $|\psi_1\rangle$ to introduce more complex quantum correlations, generating a new quantum state $|\psi_1'\rangle = V(\theta_1)|\psi_1\rangle = V(\theta_1)U(\mathbf{e}_1)|\psi_0\rangle$. Here, $V(\theta_1)$ is a variational layer controlled by the parameter θ_1.
- Repeat steps 2 and 3, encoding $(\vec{batch}_2, ..., \vec{batch}_p)$ into the respective quantum states, and performing unitary operation $V(\theta_i)$ after each encoding, where $i \in 1, 2, ..., p$. The resulting series of quantum states are $|\psi_2\rangle, ..., |\psi_p\rangle$ and their variational states $|\psi_2'\rangle, ..., |\psi_p'\rangle$. For example, for $\vec{batch}_2$, the encoded state is

$$\begin{aligned} |\psi_2\rangle &= U(\vec{batch}_2)|\psi_1'\rangle \\ &= U(\vec{batch}_2)V(\theta_1)U(\vec{batch}_1)|\psi_0\rangle \end{aligned}$$

  then applying the unitary operation of the variational layer $V(\theta_2)$ results in $|\psi_2'\rangle = V(\theta_2)|\psi_2\rangle = V(\theta_2)U(\vec{batch}_2)V(\theta_1)U(\vec{batch}_1)|\psi_0\rangle$.
- Finally, perform the measurement operation M on the quantum state $|\psi_p'\rangle$, compute the expectation value of the measurement result, which is usually obtained by random sampling in the Pauli basis. The expectation E can be represented as $E = \langle\psi_p'|M|\psi_p'\rangle$, and the obtained expectation value is used for subsequent calculations.

B. BATCH UPLOADING QUANTUM LSTM

Similar to QLSTM, we replace the classic neural network in LSTM with the aforementioned BUQNN. Fig.4 shows the BUQLSTM network we proposed, which consists of 4 BUQNNs. The expectation values output by BUQNN are combined in the LSTM network after passing through nonlinear activation functions, such as tanh and sigmoid, to update the values of various gating units. The calculation process of the 4 BUQNN units is as follows:

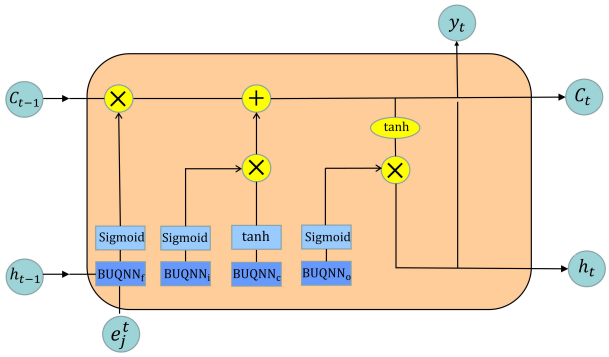


FIGURE 4. Our Proposed BUQLSTM.

$$f_t = \sigma(BUQNN_f); \tag{2}$$
$$i_t = \sigma(BUQNN_i); \tag{3}$$
$$\widetilde{c_t} = \tanh(BUQNN_c); \tag{4}$$
$$c_t = f_t * c_{t-1} + i_t * \widetilde{c_t}; \tag{5}$$
$$o_t = \sigma(BUQNN_o); \tag{6}$$
$$h_t = o_t * \tanh(c_t). \tag{7}$$

The BUQLSTM that we propose uses four BUQNN networks, represented by $BUQNN_n(n \in f, i, c, o)$. Through the above calculations, the LSTM network can obtain the hidden state h_t and cell state c_t at time step t.

Algorithm 1 outlines the numerical computation process of BUQLSTM. Initially, within each gate unit, the input vector "inputs" is partitioned into several "batch" vectors. Subsequently, these vectors are sequentially embedded into the quantum circuit following the encoding-variation order. The "weights" will be updated as part of the subsequent optimization process. Finally, the expectation values of the PauliZ operators on the relevant qubits are calculated, and the results are returned in the form of a list for further computations. Here is the explanation for the 4 BUQNN modules used in BUQLSTM:

$BUQNN_f$: $BUQNN_f$ obtains the vector f_t by combining a sigmoid function and maps the expectation value to the interval [0,1]. f_t is a crucial component of the BUQLSTM network, with its output shown in (2). It operates on c_{t-1} based on $f_t * c_{t-1}$, meaning it decides whether to "forget" or "retain" the corresponding elements from the previous cell state c_{t-1}. For instance, values of 1 or 0 indicate that the corresponding elements will be entirely retained (forgotten). Typically, the f_t vector affects cell state values between 0 and 1, indicating that only part of the information will be retained. Its function is vital for learning and modeling time dependence.

$BUQNN_i$ and $BUQNN_c$: Firstly, the $BUQNN_i$ module processes the input data v_t and outputs a set of values between 0 and 1 through a Sigmoid function, determining which information can be added to the current cell state. Simultaneously, the $BUQNN_c$ module also processes the

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903

same input data and generates a new cell state candidate $\widetilde{c_t}$ through a tanh function (as shown in (4)). Equation (5) combines the output of the input gate with the forget gate f_t, the cell state of the previous moment c_{t-1}, and the new cell state candidate $\widetilde{c_t}$, and the resulting vector will be used to update the current cell state. In other words, the output of the input gate (a real number between 0 and 1) determines how much of the new information $\widetilde{c_t}$ is added to the current cell state c_t. This mechanism allows LSTM to better remember long-term dependencies, avoiding the vanishing gradient problem of ordinary RNNs.

$BUQNN_o$: The goal of $BUQNN_o$ is to generate the output of the cell. In Equation (6), O_t obtains its output through the sigmoid function after obtaining the expectation value from $BUQNN_o$. In Equation (7), the output o_t is multiplied element-wise with the output of the update gate c_t (which is processed through a tanh activation function), generating the new hidden state vector h_t, which will be passed to the next time step for calculation.

Algorithm 1 An algorithm for BUQLSTM.

BUQLSTM(INPUT_SIZE, HIDDEN_SIZE)
inputs = $concatenate$(input_size, hidden_size)
forget gate : device$_f$ = $device$(backend, wires = w_f)
#circuit_forget(inputs, weights):
  split inputs into p batches
  for batch in p:
    $encoding$(batch, wires = w_f)
    $variation$(weights$_f$, wires = w_f)
  return ([$Expectation(PauliZ$(wire) for each wire])
input gate : device$_i$ = $device$(backend, wires = w_i)
#circuit_input(inputs, weights):
  split inputs into p batches
  for batch in p:
    $encoding$(batch, wires = w_i)
    $variation$(weights$_i$, wires = w_i)
  return ([$Expectation(PauliZ$(wire) for each wire])
update gate : device$_c$ = $device$(backend, wires = w_c)
#circuit_update(inputs, weights):
  split inputs into p batches
  for batch in p:
    $encoding$(batch, wires = w_c)
    $variation$(weights$_c$, wires = w_c)
  return ([$Expectation(PauliZ$(wire) for each wire])
output gate : device$_o$ = $device$(backend, wires = w_o)
#circuit_output(inputs, weights):
  split inputs into p batches
  for batch in p:
    $encoding$(batch, wires = w_o)
    $variation$(weights$_o$, wires = w_o)
  return ([$Expectation(PauliZ$(wire) for each wire])

C. BATCH UPLOADING QUANTUM GRU

Above, we have detailed the structure of BUQLSTM. Similar in principle to BUQLSTM, in this section, we only provide a brief introduction to the BUQGRU network. Fig.5 depicts our BUQGRU network, where we replace the classical neural network of the GRU model with BUQNN. Compared to BUQLSTM, it only requires three BUQNNs. The current input e_j^t and the previous moment's h_{t-1} are fed into the network. The reset gate, composed of $BUQNN_r$ and the sigmoid activation function, determines how much of the previous moment's hidden state information should be used when calculating the current candidate hidden state. The update gate, comprised of $BUQNN_z$ and the sigmoid function, determines how much of the previous moment's hidden state information should be preserved when calculating the current hidden state. Next, $BUQNN_z$ combined with the tanh activation function is used to calculate the candidate hidden state $\widetilde{h_t}$. Finally, equation (11) decides whether the new hidden state h_t should fully accept the candidate hidden state, retain the previous moment's hidden state, or be a compromise between the two.

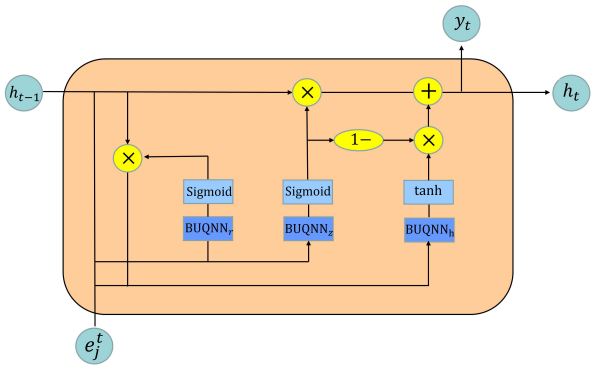


FIGURE 5. Our Proposed BUQGRU.

$$r_t = \sigma(BUQNN_r); \tag{8}$$
$$z_t = \sigma(BUQNN_z); \tag{9}$$
$$\widetilde{h_t} = \tanh(BUQNN_h); \tag{10}$$
$$h_t = (1 - z_t) * h_{t-1} + z_t * \widetilde{h_t}; \tag{11}$$

Here, $BUQNN_i, (i \in r, z, h)$ represent the reset gate circuit, update gate circuit, and candidate hidden state circuit, respectively. It is important to note that, due to the characteristics of the GRU network, the input to $BUQNN_h$ differs from other $BUQNN$. In the computation of Equation (10), the output r_t of the reset gate is multiplied by the hidden state output h_{t-1} from the previous time step. This product determines how much information from the previous time step can be utilized. The resulting product is concatenated with e_j^t and serves as input to $BUQNN_h$. After passing through the tanh activation function, a new candidate hidden state is obtained.

D. PARAMETER NON-SHARING BUQNN

In this section, we will introduce the proposed PN-BUQNN (Parameterized-Nonshared BUQNN) to enhance the learning capability of BUQNN. As shown in Fig.6.(a), in traditional

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903

QLSTM networks, the linear embedding layers before each VQC and after each VQC are non-shared parameters. While this reduces the parameter count of the model, it may introduce some issues. By using the same linear transformation for all variational quantum circuits, the model's ability to learn different input data features could be limited. For instance, if the forget gate and the input gate need to extract different features from the input data, a model with shared parameters may struggle to simultaneously satisfy the requirements of both gates.Furthermore, during the training process, all variational quantum circuits backpropagate through the same linear layer. This can lead to gradient explosion or vanishing gradients. If the gradient of a particular variational quantum circuit is exceptionally large, it may "overwhelm" the gradients of other circuits, making it difficult for the entire network to learn effectively.To address the aforementioned issues, we propose PN-BUQNN, as depicted in Fig.6.(b).

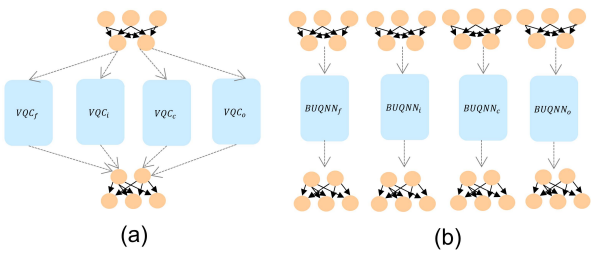


FIGURE 6. (a) represents a parameter-shared VQC, while (b) represents our proposed PN-BUQNN.

We employ linear layers before and after each BUQNN, which means they can learn more appropriate input and output transformations for themselves. The independent linear layers imply that each BUQNN can learn and extract different features, with optimizations being specific to each BUQNN. During the training process, since each BUQNN has its own linear layer, their gradient updates will no longer affect each other. Furthermore, adding a linear layer before $BUQNN$ can to some extent alleviate the issue of gradient vanishing. The issue of gradient vanishing is discussed in low-qubit Variational Quantum Circuits (VQC) [35], and a similar problem exists in the encoding layer of $BUQNN$. The input feature vector v_t^i generates different rotation angles for the parameters of Ry and Rz gates using the arctan function. For the input $\vec{batch}1$, the encoding layer of $BUQNN$ can be represented as: $encoding(\vec{batch}_1) = R_z(\arctan(v_t^1)^2)R_y(\arctan(v_t^1))\,|0\rangle \ldots R_z(\arctan(v_t^N)^2)\,R_y(\arctan(v_t^N))\,|0\rangle$. It can be observed that the encoding layer of BUQNN involves the use of arctan, whose derivative may lead to gradient vanishing. For instance, the derivative of $R_y(\arctan(v_t^1))$ is given as follows:

$$\frac{\partial R_y\left(\arctan\left(v_t^1\right)\right)}{\partial w_i} = -R_y\left(\arctan\left(v_t^1\right)\right)^T \frac{\partial \arctan\left(v_t^1\right)}{\partial w_i} \tag{12}$$

$$\frac{\partial \arctan\left(v_t^1\right)}{\partial w_i} = \frac{1}{1+\left(v_t^1\right)^2}\frac{\partial v_t^1}{\partial w_i} \tag{13}$$

Here, v_t^1 is an element in $\vec{batch}_1$, and w_i denotes the updatable rotation gate parameters. From (13), it is evident that the value of v_t^1 should not be too large or too small, as it may lead to gradient vanishing. Therefore, a clip operation on the output can be added after the linear layer preceding BUQNN, restricting the output within a specified range. Through this approach, it is possible to optimize the weights of the linear transformation effectively, thus mitigating gradient vanishing to a certain extent. In subsequent experiments, we will use such a linear layer to control the output within the range of [-3, 3].

IV. DATASET AND EXPERIMENTAL RESULT

A. DATASET

The experiments utilized two Bengali text classification datasets. The BOOK-Reviews dataset [36] is a collection of Bengali book reviews gathered from the internet (such as blogs, Facebook, and e-commerce websites). This dataset is a binary classification dataset (with positive and negative classes) containing 2000 book reviews. The other dataset used in the experiment is YouTube-B [37], a collection of comments on Bengali dramas collected from the YOUTUBE site. It contains 11807 comments, of which 8500 are positive and 3307 are negative. Given the limitations of current NISQ devices on the efficiency of quantum algorithms in classification tasks, we selected 2000 entries from the YouTube-B dataset for the experiment, with 1700 for training and 300 for testing. We named the modified YouTubeB dataset as YouTubeB-S. The average number of words per sentence in the BOOK-Reviews and YouTubeB-S datasets is 46 and 21, respectively. As shown in Table 1, to prevent long-tail distribution in the data, the number of data entries in each category was kept approximately equal. The dataset we used is available at https://github.com/nuistyl/Bengali-dataset.

TABLE 1. The datasets used in the experiment are the BOOK-Reviews dataset and the YouTubeB-S dataset

Dataset	Positive	Negative	All
BOOK-Reviews	996	1004	2000
YouTubeB-S	1005	995	2000

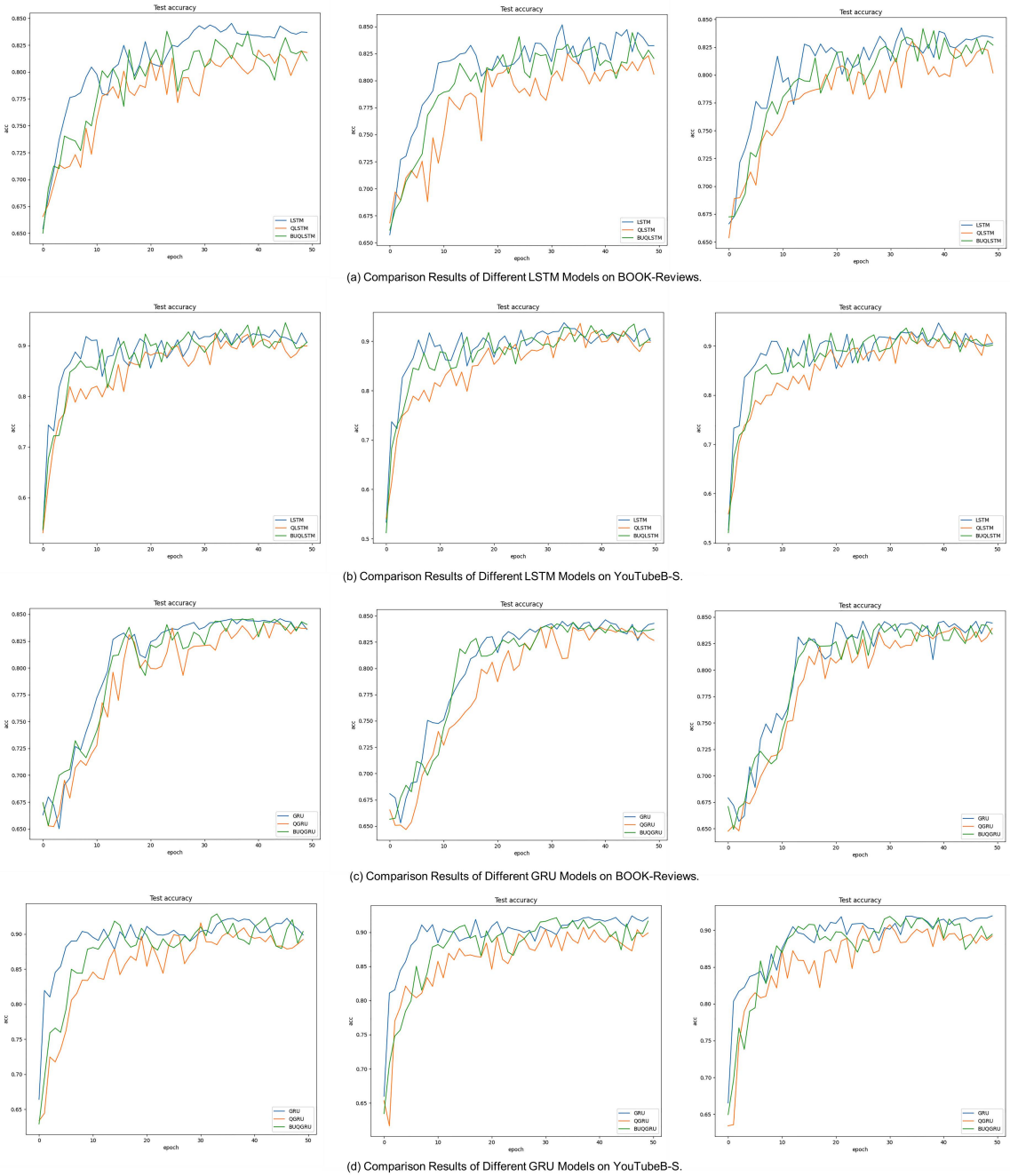
TABLE 2. Comparison of the Number of Model Parameters Used

Model	Number of Quantum Gates	Number of Classical Parameters	All
LSTM	0	224	224
GRU	0	168	168
QLSTM	96	72	168
QGRU	72	72	144
BUQLSTM	144	20	164
BUQGRU	108	20	128

In the subsequent experimental process, we will use these

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903



(a) Comparison Results of Different LSTM Models on BOOK-Reviews.

(b) Comparison Results of Different LSTM Models on YouTubeB-S.

(c) Comparison Results of Different GRU Models on BOOK-Reviews.

(d) Comparison Results of Different GRU Models on YouTubeB-S.

FIGURE 7. Experimental results graph.

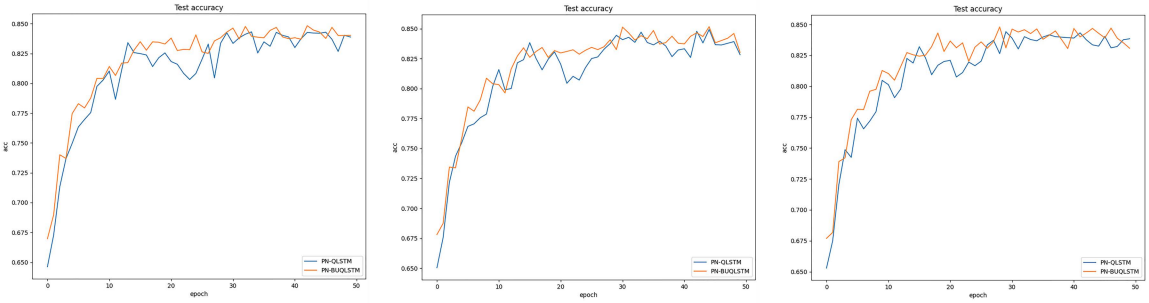
two datasets to test the two structures shown on both sides of Fig.2, respectively.

B. STRUCTURE WITH BUQRNN

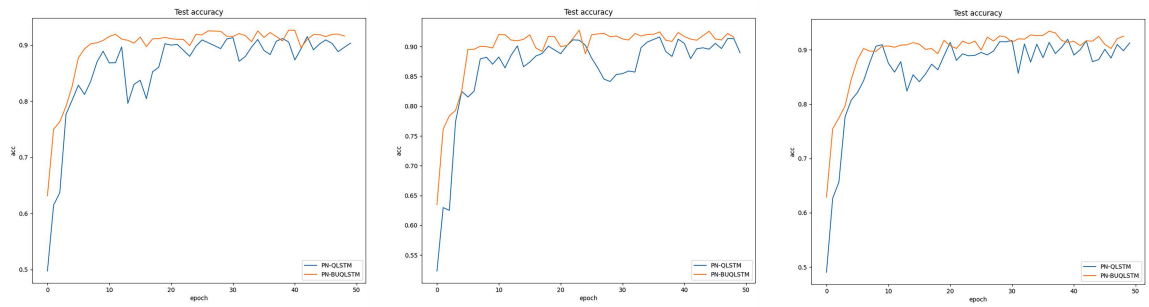
In this section, we conducted experiments on the BUQRNN architecture shown in Fig.2. We employed BUQNN to define BUQLSTM and BUQGRU and conducted experiments using a 4-qubit circuit. For comparison, we also constructed a traditional variational quantum circuit with 4 qubits. In Equation (1), the input vector is passed through the MBERT model, resulting in the embedding vector e_j^t. To facilitate quantum simulation, we combined MBERT with a linear layer to control the feature size to 8 dimensions. We also compared it with a classical LSTM network with an input size of 8 dimensions and 224 parameters. All experiments used a hidden layer size of 4 dimensions, and the feature dimensions combined with the hidden layer dimensions were fed into the model. For a fair comparison, the depth of the VQC in the traditional QRNN was set to 2. Reference [38] mentions that when the architecture of the VQC is extensive when the

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

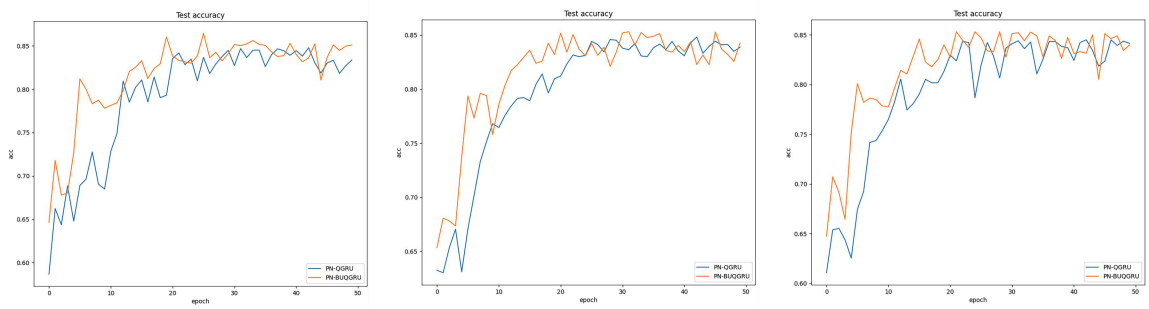
This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903



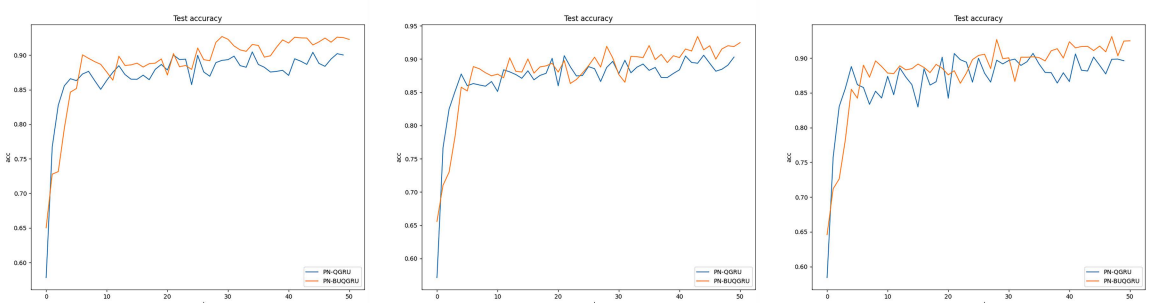
(a) Comparison Results of LSTM Models with Parameter Non-Sharing on BOOK-Review



(b) Comparison Results of LSTM Models with Parameter Non-Sharing on YouTubeB-S.



(c) Comparison Results of GRU Models with Parameter Non-Sharing on BOOK-Review.



(d) Comparison Results of GRU Models with Parameter Non-Sharing on YouTubeB-S.

FIGURE 8. Experimental results graph.

circuit depth does not decrease the number of gates - the quantum data classification errors of VQC typically decrease exponentially with the increase in circuit depth. This rapid error suppression ends when reaching the final Helstrom limit of quantum state discrimination. However, considering the limitations of NISQ devices, it is challenging to train more parameters. In future work, we plan to further explore the impact of VQC depth on classification accuracy by improving experimental design and adopting more advanced quantum devices. The model parameter counts used in the experiments are shown in Table 2. The experiments were conducted using the ADAMW optimizer and the cross-entropy loss function. The PENNYLANE framework was used for modeling the quantum circuits, which includes multiple built-in simulators to meet different task requirements.

All the aforementioned experiments were conducted under the conditions of a learning rate of 0.01, 50 epochs, and the use of linear warm-up optimization method. To enhance the persuasiveness of the experimental results, we conducted 10 experiments for each trial using datasets divided into

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903

different partitions to obtain the mean accuracy. Due to space constraints, we present only the first three results for each experiment. Fig.7 presents the comparison results of different models on two datasets, where the highest accuracy is shown in Table 3. Overall, while quantum structures have disadvantages compared to classical structures, the BUQRNN-based structures show improved accuracy compared to traditional QRNN structures on both datasets. This is because traditional QRNN utilizes linear layers to reduce the dimensionality of the input feature sequence during circuit simulation, which may result in a loss of semantic information to some extent.

TABLE 3. Accuracy of Structures with BUQRNN and Baseline Structures on BOOK-Reviews and YouTubeB-S Validation Sets

Model	BOOK-Reviews	YouTubeB-S
MBERT+LSTM	84.231	92.394
MBERT+QLSTM	82.235	91.266
MBERT+BUQLSTM	**83.121**	**92.037**
MBERT+GRU	84.524	92.214
MBERT+QGRU	83.857	90.735
MBERT+BUQGRU	**84.227**	**91.728**

In contrast, BUQRNN adopts the approach of uploading the complete sequence into the circuit in batches. On the YOUTUBE dataset, the BUQLSTM structure and the BUQGRU structure achieve improvements of 0.771% and 0.993%, respectively, compared to the QLSTM structure and the QGRU structure. On the BOOK-REVIEWS dataset, the BUQLSTM structure and the BUQGRU structure achieve improvements of 0.886% and 0.370%, respectively, compared to the QLSTM structure and the QGRU structure. Although the improvements in accuracy are limited, our proposed BUQRNN model has fewer parameters compared to QRNN and classical RNN, making it more suitable for low-resource language domains.

C. STRUCTURE WITH PN-BUQRNN

In the previous section, we presented the results of BUQRNN. In this section, we tested the structure equipped with PN-BUQRNN, in which independent linear layers were used before and after each BUQNN. For comparative experiments, to maintain consistency with the structure of PN-BUQRNN, we set the depth of the VQC in traditional QRNN to 3 and also used independent linear layers before and after each VQC, denoted as PN-QRNN.

The experimental settings remained consistent with the previous section. Fig.8 illustrates the comparison results of using the parameter non-shared LSTM model and the GRU model on the two datasets. The highest accuracy is shown in Table 4. Due to space constraints, we present only the results of the first three runs for each experiment. Overall, the PN-BUQRNN structure outperforms the classical RNN structure and the PN-QRNN structure on both datasets. After applying the parameter non-shared VQC, the PN-QRNN structure even performs worse in certain tasks. This is due to the information loss caused by the dimensionality reduction of the linear layer, which prevents it from working effectively. In contrast, in the PN-BUQRNN structure, we fully load the information into the BUQNN circuit and optimize each BUQNN circuit independently, resulting in better results.

TABLE 4. The accuracy of the structure with PN-BUQRNN compared to the baseline structure on the YouTubeB-S and BOOK-Reviews validation sets

Model	BOOK-Reviews	YouTubeB-S
MBERT+LSTM	84.231	92.394
MBERT+PN-QLSTM	84.266	91.416
MBERT+PN-BUQLSTM	**84.738**	**92.681**
MBERT+GRU	84.524	92.214
MBERT+PN-QGRU	84.343	90.762
MBERT+PN-BUQGRU	**85.304**	**92.690**

D. ABLATION STUDY ON PARAMETER NON-SHARING CIRCUITS

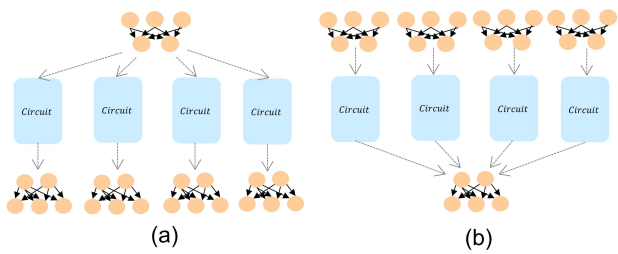


FIGURE 9. There are two modes for parameter non-shared circuits.

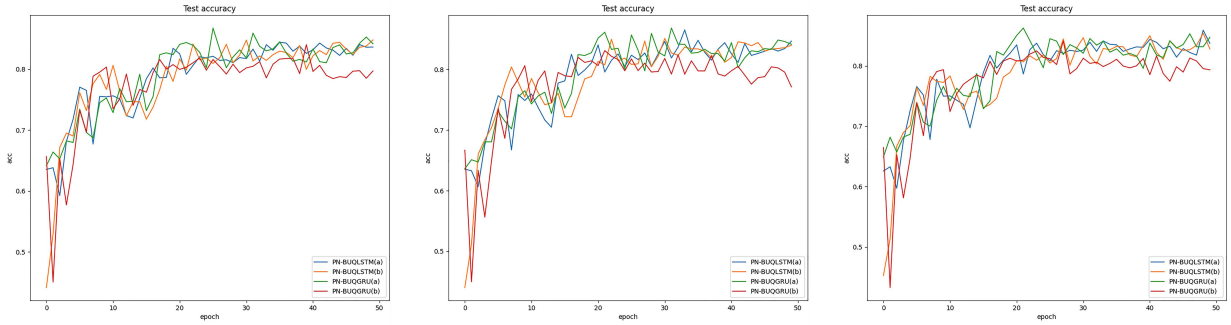
In previous experiments, we constructed models with parameter non-sharing by using separate linear layers before and after each quantum circuit. However, there is still a question to be verified. As shown in Fig.9, what would be the result if we only use parameter non-sharing linear layers at the input end of the quantum circuit, or only at the output end?

We tested the PN-BUQRNN structure with two modes shown in Fig.9.(a) and Fig.9.(b). Fig.10 and Table 5 present the experimental results, indicating a decreasing trend in accuracy for both modes on the two datasets. Among them, mode (a) performs better than mode (b). When using mode (a), the accuracy slightly decreases compared to the case where both the input and output ends use parameter non-shared in the previous section, while mode (b) significantly affects the experimental results. This verifies the correctness of our structure, which is that it is necessary to add parameter non-shared linear layers at both the input and output ends of the quantum circuit, and it will better optimize the quantum circuit.

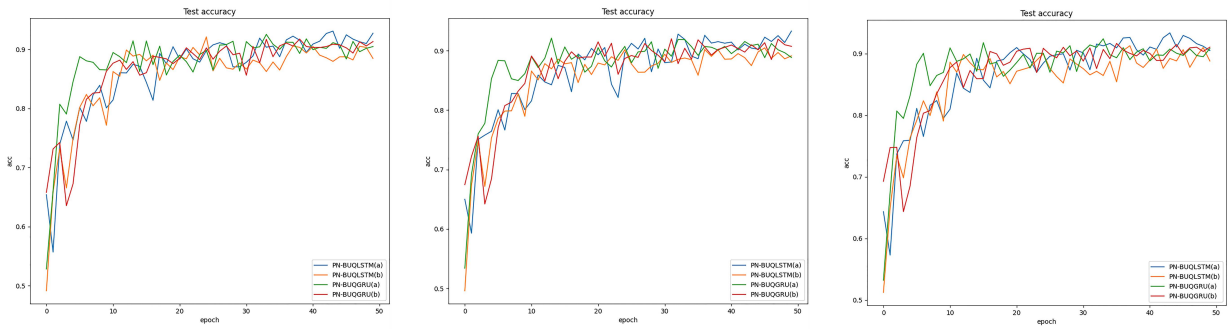
This confirms the correctness of our structure i.e., adding parameter non-sharing linear layers at both the input and output ends of the quantum circuit is necessary and will better optimize the quantum circuit. Therefore, we believe

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903



(a) The comparison results of the above two methods on the BOOK-Reviews.



(b) The comparison results of the above two methods on the YouTubeB-S.

FIGURE 10. The experimental results of the two modes are presented on the datasets BOOK-Reviews and YouTubeB-S, respectively.

that it is possible to select the appropriate model structure based on different task requirements. That is, if accuracy is pursued, the BUQRNN with parameter non-sharing linear layers before and after can be used. If both accuracy and efficiency are pursued, it is also feasible to only use the parameter non-sharing structure after BUQNN.

TABLE 5. PN-BUQRNN Structure with Two Types of Parameter-Unshared Circuits, where superscript (a) refers to a in Fig.9 and superscript (b) refers to b in Fig.9

Model	BOOK-Reviews	YouTubeB-S
PN-BUQLSTM$^{(a)}$	**84.547**	**92.036**
PN-BUQGRU$^{(a)}$	**85.047**	**91.644**
PN-BUQLSTM$^{(b)}$	84.232	89.925
PN-BUQGRU$^{(b)}$	82.435	91.356

V. CONCLUSION

To address the brute force approach taken by traditional QRNNs when handling feature dimensions larger than the number of qubits, we propose a solution that breaks feature vectors into batches and passes them through the circuit, thereby increasing the available information. As such, in this paper, we design a novel incremental quantum neural network, termed BUQNN, and apply it to LSTM and GRU networks, forming BUQRNN. Experimental results on the Bengali corpus demonstrate that, compared to traditional QRNNs, our proposed BUQRNN improves accuracy by up to 0.993% while reducing model complexity on average by 12%. Given that traditional QRNNs are unable to independently optimize the linear layers before and after the quantum circuit, thereby limiting their adaptability to VQC, we are inspired to combine the aforementioned BUQNN design with a recurrent neural network with non-shared parameters, resulting in a model class called PN-BUQRNN. Quantum neural networks constructed using this approach perform better in experiments, surpassing both classical neural networks and traditional QRNNs on two Bengali text datasets. As an attempt in the field of low-resource language quantum neural networks, we demonstrate the feasibility of applying quantum algorithms to address practical issues in the low-resource text domain. Considering the limited computational resources in low-resource regions, our method allows for circuit simulation with a S number of qubits, aligning with the characteristics of the low-resource language domain. Finally, our goal is to introduce our proposed model to natural language processing tasks in more low-resource regions, in order to address a broader range of real-world issues.

VI. APPENDIX

A. FEASIBILITY EXPLORATION OF BUQRNN AS A WORD EMBEDDING MODEL

In the discussion of the aforementioned related work, reference [27] mentions an approach that utilizes a quantum neural network as a word embedding model. The authors employed QLSTM as the pretraining model and then utilized the obtained word embeddings for downstream tasks. Due to the similarity with this work, in this section, we

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903

explore the applicability of the proposed BUQRNN as an embedding layer model on Bengali text corpora. The experimental data consists of the aforementioned BOOK-Reviews and YouTubeB-S datasets, and the specific methodology is outlined below:

TABLE 6. The results of word vector representations obtained using BUQRNN and QRNN on the BOOK-Reviews and YouTubeB-S datasets.

Model	BOOK-Reviews	YouTubeB-S
QLSTM	76.314	82.246
QGRU	77.241	82.436
BUQLSTM	**77.243**	**83.357**
BUQGRU	**78.923**	**83.143**

- Pretraining Quantum Models: We selected the proposed BUQRNN and QRNN mentioned in Section IV.B as comparative models. To ensure a fair comparison, we kept the experimental parameters consistent with those in Section IV.B, with word embedding sizes of 8 for both BUQRNN and QRNN, and vocabulary size as the output size.
- Pretraining: The aforementioned models were separately trained on the BOOK-Reviews and YouTubeB-S until convergence. During the language model training, sentiment labels were disregarded.
- Model Evaluation: We assessed the utility of the pretrained embedding vectors in downstream SA tasks. Specifically, we used the pretrained word embeddings from BUQRNN and QRNN to train a linear layer-based classifier. The choice of a linear layer as the classifier aimed for experimental convenience. Consistent with the discussed methods in related work, we employed a random subset of language model training data for evaluation to avoid introducing additional noise into the model.

We conducted ten experiments to obtain the mean performance, and Table 6 presents our experimental results.Table 6 presents our experimental results. The word embeddings trained by BUQRNN demonstrate higher accuracy compared to QRNN, indicating the high utility of our proposed BUQRNN as a word embedding model in the low-resource language domain. The primary reason for BUQRNN's superior results lies in its ability to fully load input vectors into the circuit, thus avoiding the loss of semantic information. This experimental outcome also reinforces our viewpoint discussed earlier. In future explorations, we anticipate using BUQRNN as a word embedding model for various downstream tasks in the low-resource language domain.

B. THE DIFFERENCE BETWEEN BUQNN AND VQC USING AMPLITUDE ENCODING

In VQC, the encoding layer can utilize various encoding methods, such as the most common angle encoding or amplitude encoding. In BUQNN, we adopt angle encoding to encode feature vectors into the quantum circuit. However, can amplitude encoding be used? The nature of BUQNN (essentially a VQC using batch uploading) makes it difficult to use amplitude encoding for feature encoding; otherwise, BUQNN would degrade into a traditional VQC circuit. To compare it with amplitude encoding, we used the BUQLSTM in Section IV.B and created a QLSTM using amplitude encoding. For 8-dimensional input data, amplitude encoding requires 4 qubits to encode the features into the circuit. The model layers and experimental hyperparameters were kept consistent with the above Section IV.B.

TABLE 7. Comparison of our BUQLSTM network with amplitude-encoded QLSTM network on the BOOK-Reviews and YouTubeB-S datasets.

Model	BOOK-Reviews	YouTubeB-S
Amplitude-QLSTM	82.579	91.557
BUQLSTM	83.121	92.037

Table 7 demonstrates the advantages of the proposed BUQLSTM network. To some extent, amplitude encoding theoretically offers higher information capacity. However, in practical applications, the state preparation required for amplitude encoding is expensive in terms of operations. In contrast, the encoding method of BUQLSTM is more direct and requires only a small number of qubits to embed feature vectors into the circuit.

REFERENCES

[1] J. J. Grefenstette, "Genetic algorithms and machine learning,"in Proceedings of the sixth annual conference on Computational learning theoryz.,Santa Cruz., CA.USA,1993,pp.3–4.
[2] T.M. Mitchell, Machine learning, NY,USA:McGraw-hill,2007,pp.14–16.
[3] Z. Zhou, Machine learning, USA:Springer Nature,2021,pp.5–25.
[4] P. M. Nadkarni, L. Ohno-Machado, and W. W. Chapman, "Natural language processing: an introduction," J AM MED INFORM ASSN, vol. 18, no. 5, pp.544–551,Sep,2011,doi:10.1136/amiajnl-2011-000464.
[5] D. W. Otter, J. R. Medina, and J. K. Kalita, "A survey of the usages of deep learning for natural language processing," IEEE TNNLS, vol. 32, no. 2, pp.604–624,Apr,2020,doi:10.1109/TNNLS.2020.2979670.
[6] W. Medhat, A. Hassan, and H. Korashy, "Sentiment analysis algorithms and applications: A survey,"AIN SHAMS ENG J, vol. 5, no. 4, pp.1093–1113,Apr,2014,doi:10.1016/j.asej.2014.04.011.
[7] M. Taboada, J. Brooke, M. Tofiloski, K. Voll, and M. Stede, "Lexicon-based methods for sentiment analysis," Computational linguistics, vol. 37, no. 2, pp.267–307, June, 2011, doi:10.1162/COLI_a_00049.
[8] A. Hasan, S. Moin , A. Karim, and S. Shamshirband, "Machine learning-based sentiment analysis for twitter accounts," Mathematical and computational applications, vol. 23, no. 1, pp.11, Oct,2018, doi:10.3390/mca28050101.
[9] B. Agarwal, R. Nayak, N. Mittal, and S. Patnaik, Deep learning-based approaches for sentiment analysis,USA:Springer, 2020,pp.2–31.
[10] Zhang, Binlong and Zhou, Wei, "Transformer-Encoder-GRU (TE-GRU) for Chinese Sentiment Analysis on Chinese Comment Text," Neural Processing Letters, vol. 55, no.2, pp.1857–1867, July,2022, doi: 10.1007/s11063-022-10966-8.
[11] Luo, Xiaoyu, "Efficient English text classification using selected machine learning techniques," Alexandria Engineering Journal, vol.60, no.3, pp.3401–3409, Feb,2021, doi:10.1016/j.aej.2021.02.009.
[12] Min B, Ross H, Sulem E, et al, " Recent advances in natural language processing via large pre-trained language models: A survey," ACM Computing Surveys, vol.56, No. 30, pp.1–40, Sep,2023,doi:10.1145/3605943.
[13] Haifeng Wang, Jiwei Li, Hua Wu, Eduard Hovy, Yu Sun, " Pre-Trained Language Models and Their Applications," Engineering, vol.25, pp.51–65, Apr,2022, doi:10.1016/j.eng.2022.04.024.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4

This article has been accepted for publication in IEEE Transactions on Quantum Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TQE.2024.3373903

[14] Alchieri, Leonardo and Badalotti, Davide and Bonardi, Pietro and Bianco, Simone, " An introduction to quantum machine learning: from quantum logic to quantum deep learning," Quantum Machine Intelligence, vol.3, pp.1–30, Oct,2021, doi:10.1007/s42484-021-00056-8.

[15] Wiebe, Nathan and Kapoor, Ashish and Svore, Krysta M," Quantum deep learning," 2014, arxiv:1412.3489.

[16] Bennett, Charles H and Bernstein, Ethan and Brassard, Gilles and Vazirani, Umesh, " Strengths and weaknesses of quantum computing," SIAM journal on Computing, vol.26, no.5, pp.1510–1523, Jan,1997, doi:10.1137/S0097539796300933.

[17] Lai, Wei, Jinjing Shi, and Yan Chang, "Quantum-Inspired Fully Complex-Valued Neutral Network for Sentiment Analysis," Axioms, vol. 12, no. 3, pp.308, Feb, 2023, doi:10.3390/axioms12030308.

[18] S. Y. Chen, S. Yoo, and Y. L. Fang, "Quantum long short-term memory," in ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2022, pp.8622–8626.

[19] Preskill, John, "Quantum computing in the NISQ era and beyond," Quantum, vol.2, pp.79, Aug,2018, doi:10.22331/q-2018-08-06-79.

[20] Y. Cao, X. Zhou, X. Fei, H. Zhao, W. Liu, and J. Zhao, "Linear-layer-enhanced quantum long short-term memory for carbon price forecasting,"Quantum Machine Intelligence, vol. 5, no. 2, pp.1–12, Jul, 2023, doi:10.1007/s42484-023-00115-2.

[21] S. Sazzed, âĂIJCross-lingual sentiment classification in low-resource bengali language," in Proceedings of the sixth workshop on noisy user-generated text (W-NUT 2020), Nov, 2020, pp.50–60.

[22] C. -H. H. Yang, J. Qi, S. Y. -C. Chen, Y. Tsao and P. -Y. Chen, "When BERT Meets Quantum Temporal Convolution Learning for Text Classification in Heterogeneous Computing," ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Singapore, Singapore, 2022, pp. 8602-8606, doi: 10.1109/ICASSP43922.2022.9746412.

[23] Al-Amin M, Islam M S, Uzzal S D, "Sentiment analysis of Bengali comments with Word2Vec and sentiment information of words" in 2017 international conference on electrical, computer and communication engineering (ECCE), Feb, 2017, pp.186-190.

[24] Chowdhury, Pallab and Eumi, Ettilla Mohiuddin and Sarkar, Ovi and Ahamed, Md Faysal, âĂIJBangla news classification using GloVe vectorization, LSTM, and CNN" in Proceedings of the International Conference on Big Data, IoT, and Machine Learning: BIM 2021, Dec,2017, pp.723-731.

[25] Hossain, Md Rajib and Hoque, Mohammed Moshiul and Sarker, Iqbal H, Text Classification Using Convolution Neural Networks with FastText Embedding, USA:Springer 2021, pp.101–113.

[26] Pires, Telmo and Schlinger, Eva and Garrette, Dan, âĂIJHow multilingual is multilingual BERT?," 2019, arxiv:1906.01502.

[27] S. S. Li et al., "PQLM - Multilingual Decentralized Portable Quantum Language Model," ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Rhodes Island, Greece, 2023, pp. 1-5, doi: 10.1109/ICASSP49357.2023.10095215.

[28] R. Di Sipio, J. -H. Huang, S. Y. -C. Chen, S. Mangini and M. Worring, "The Dawn of Quantum Natural Language Processing," ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Singapore, Singapore, 2022, pp. 8612-8616, doi: 10.1109/ICASSP43922.2022.9747675.

[29] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and others, "Variational quantum algorithms," Nature Reviews Physics, vol. 3, no. 9, pp.625–644, Aug, 2021, doi:10.1038/s42254-021-00348-9.

[30] R. Horodecki, P. Horodecki, M. Horodecki, and K. Horodecki, "Quantum entanglement," Reviews of modern physics, vol. 81, no. 2, pp.865, June, 2009, doi:10.1103/RevModPhys.81.865.

[31] Ryan LaRose and Brian Coyle, "Robust data encodings for quantum classifiers,"Physical Review A, vol. 103, no. 2, Aug2020, doi:10.1103/PhysRevA.102.032420.

[32] I. Glendinning, "The bloch sphere," in QIA Meeting, 2005, pp.3–18.

[33] M. Periyasamy, N. Meyer, C. Ufrecht, D. D. Scherer, A. Plinge, and C. Mutschler, "Incremental data-uploading for full-quantum classificationn," in 2022 IEEE International Conference on Quantum Computing and Engineering (QCE), May, 2022, pp.31–3.

[34] Perez-Salinas, Adrian, et al. "Data re-uploading for a universal quantum classifier." Quantum, vol.4, pp.226, Feb,2020, doi:10.22331/q-2020-02-06-226.

[35] Z. Hong, J. Wang, X. Qu, C. Zhao, W. Tao and J. Xiao, "QSpeech: Low-Qubit Quantum Speech Application Toolkit," 2022 International Joint Conference on Neural Networks (IJCNN), Padua, Italy, 2022, pp. 01-08, doi: 10.1109/IJCNN55064.2022.9892496.

[36] S. Sazzed, "Cross-lingual sentiment classification in low-resource bengali language," in Proceedings of the sixth workshop on noisy user-generated text (W-NUT 2020), Nov, 2020, pp.50–60.

[37] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Y. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in Proceedings of the 2013 conference on empirical methods in natural language processing, Oct, 2013, pp.1631–1642.

[38] Zhang, Bingzhi, and Quntao Zhuang. "Fast decay of classification error in variational quantum circuits." in Quantum Science and Technology, June, 2022, doi:10.1088/2058-9565/ac70f5.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4